DESIGN DOCUMENT · IMPLEMENTED

One dashboard, many hosts

How Cubeherd splits into a Dashboard container and any number of Agent containers, so a single web interface manages Minecraft servers on several machines.

Companion to `prototype/nodes.html` · Status: implemented, see ARCHITECTURE.md · Decisions taken: the dashboard calls the agents; the dashboard can optionally keep an embedded local agent.

1. Summary

Today one process does everything: web UI, Docker access, `docker compose` calls, file access to `servers/<id>/`, the backup scheduler and the S3 upload. That ties one dashboard to exactly one host.

The proposal splits the same code base into two roles that ship in **one image** and are selected by a command:

ROLE	RESPONSIBILITY
Dashboard `cubeherd`	Web UI, login, global settings, a registry of nodes. It holds **no server data** and needs no Docker socket (unless the optional embedded local agent is on).
Agent `cubeherd agent`	Runs next to the Minecraft servers on one host. Owns Docker access, the server folders, backups and the scheduler. Exposes a small authenticated HTTP API and has no UI.

> **Core idea**
> The existing `Manager` becomes the agent's engine, unchanged in behaviour. The web layer stops calling it directly and talks to a `Node` interface instead, which has two implementations: *local* (calls the Manager in-process) and *remote* (calls an agent over HTTPS). Everything the dashboard shows is therefore "the sum of all nodes".

What stays the same

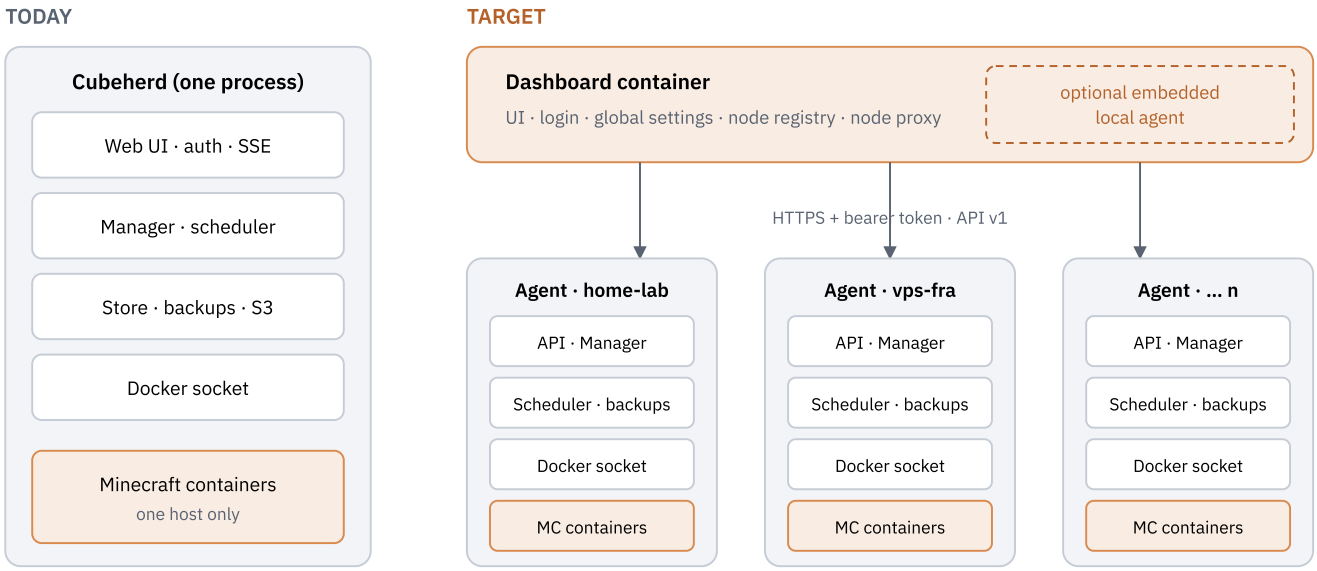
- Servers are still plain folders with `cubeherd.json` and `compose.yml`; they keep working with `docker compose` without any Cubeherd process.
- No database. No background polling: live data flows only while a page is open.
- Idle cost on each host stays around 10 MB RAM and 0 % CPU.
- One admin account on the dashboard; the UI looks and behaves as it does now.

What changes

- A server is identified by **(node, id)**; URLs become `/n/<node>/s/<id>`. Old `/s/<id>` links redirect to the local node.
- Ports only have to be unique **per node**.
- New **Nodes** page, node grouping in the sidebar and the overview, and a placement step in the create wizard.
- Global settings (S3, CurseForge key, time zone, language) stay in the dashboard and are pushed to the agents.

2. Architecture

Left: today. Right: target. Arrows show who opens the connection.



The embedded local agent is the "home-lab" node when the dashboard container also has the Docker socket.

Code layout

PACKAGE	CHANGE
`internal/node` (new)	The `Node` interface: list/get views, lifecycle, RCON, logs/stats/perf streams, files, world, backups, addons, discover/import, host info. Two implementations: `local` and `remote`.
`internal/manager`	Unchanged logic. Becomes the engine behind `node/local`. Direct `Store` path access from the web layer (`handlers.go`, `files.go`) moves behind the interface.
`internal/agent` (new)	HTTP server exposing the Node API (`/v1/…`), token check, streaming handlers.
`internal/web`	Handlers resolve `{node}` from the URL and call the matching `Node`. Templates gain node chips, grouping, the Nodes page and the placement step.
`internal/config`	Dashboard config gains the node registry. The agent has a small config of its own (token hash, pushed global settings).
`cmd/cubeherd`	Subcommand `agent`; env `CUBEHERD_AGENT_TOKEN`, `CUBEHERD_AGENT_LISTEN` (default `:8421`), `CUBEHERD_LOCAL_AGENT`.

3. Dashboard ↔ agent protocol

The dashboard opens every connection; agents never call out. That keeps agents passive (no registration state, nothing to reconnect) and makes streaming simple. The agents must be reachable from the dashboard over a LAN or VPN, as the README already requires for the dashboard itself.

Transport and trust

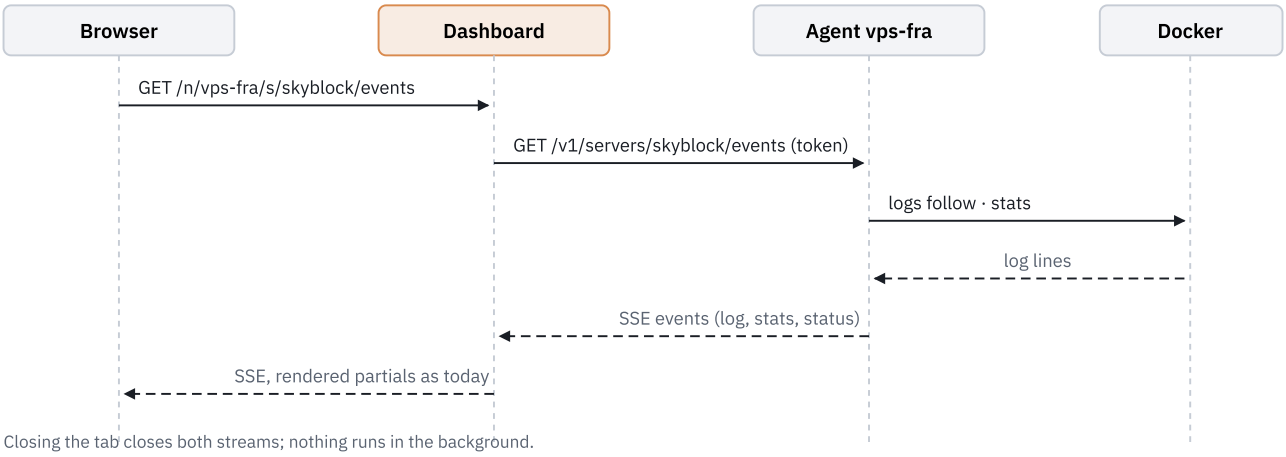
- **HTTP/JSON** over HTTPS. The agent generates a self-signed certificate on first start; the dashboard pins its SHA-256 fingerprint when the node is added (trust on first use, shown in the add-node dialog). Plain HTTP stays possible inside a VPN.
- **Bearer token** per agent, created when the node is added and passed as `CUBEHERD_AGENT_TOKEN`. The agent keeps only a hash. The token can be rotated from the Nodes page. No cookies, so no CSRF surface on the agent.
- Every call carries `X-Cubeherd-API: 1`. Agent and dashboard must share the API major; the minor/patch version may differ, and the dashboard shows an "update agent" hint.
- **Streams pass through:** SSE and chunked file, world and backup bodies are proxied byte for byte, with no buffering and no temp file on the dashboard. The world ZIP is still built on the agent.
- **Same blast radius as today:** whoever holds a token controls Docker on that host. Tokens are long random strings, compared in constant time, and the agent applies the login-throttling already used by the dashboard.

API sketch (v1)

ENDPOINT	PURPOSE
`GET /v1/info`	Version, API major, host name, Docker engine, data directory. Used by the connection test.
`GET /v1/host`	CPU, RAM, disk (what the host cards show).
`GET /v1/servers` · `POST /v1/servers`	List views; create from a definition.
`GET|PUT|DELETE /v1/servers/{id}`	Read, update settings, uninstall (`?mode=keep|delete`).
`POST /v1/servers/{id}/{start|stop|restart|update}`	Lifecycle. Returns `202` with an operation id; progress arrives on the event stream.
`GET /v1/servers/{id}/events` (SSE)	Status, stats, log lines, perf samples. Opened only while the page is open.
`POST /v1/servers/{id}/rcon`	Console command; players list.
`GET|PUT|DELETE /v1/servers/{id}/files/{path…}`	List, read, write, delete. Streamed bodies; limits and path checks stay on the agent.
`GET /v1/servers/{id}/world.zip` · `PUT …/world`	World download and import as streams.
`GET|POST|DELETE /v1/servers/{id}/backups[/{name}]`	List, run, verify, restore, download, delete.
`PUT /v1/config`	The dashboard pushes global settings (S3, CurseForge key, time zone, language).
`GET /v1/discover` · `POST /v1/import`	Adopt existing itzg containers on that host.

4. Request flows

Opening a server console



Starting a server on a remote node

1. Browser posts `/n/vps-fra/s/skyblock/start` to the dashboard (session cookie, CSRF check as today).
2. The dashboard calls `POST /v1/servers/skyblock/start` on the agent and returns the status partial immediately.
3. The agent runs `docker compose up -d` from `servers/skyblock/` and records the operation; the dashboard reads progress from the event stream.
4. If the agent is unreachable the dashboard answers with a clear "node unreachable" message and does not queue the action.

Backups and the scheduler

The scheduler runs **on each agent**, because the archives, the server images used for verification and the data are all there. Schedules live in each server's `cubeherd.json`. A dashboard outage, restart or network split therefore never skips a backup. The S3 upload also goes directly from the agent to the bucket. The dashboard only lists, triggers and downloads (streamed through).

Global settings

S3 target, CurseForge key, time zone and language are edited once in the dashboard. On every connect and after every change it sends `PUT /v1/config`; the agent persists the result (mode 0600) so it keeps working after its own restart even while the dashboard is down. The trade-off: S3 credentials are stored on each agent that uploads, which is inherent to uploading from there.

5. Who owns what

DATA	LIVES ON	NOTES
Server definitions, compose files, world data	Agent	Exactly as now; source of truth stays `servers/<id>/cubeherd.json`.
Backups, backup health, schedules	Agent	Run and evaluated locally; the dashboard only displays them.
Node registry (name, address, token, fingerprint)	Dashboard	New section in `cubeherd.json`; the token is stored like the CurseForge key (mode 0600).
Admin password, sessions, language, update notice	Dashboard	Unchanged.
Global settings (S3, CurseForge, time zone)	Dashboard, copied to agents	Pushed on connect and on change.
Bind-IP list and default port, per node	Dashboard	The host's own addresses cannot be discovered from a container, so the list is a per-node setting.
Last-known node and server state	Dashboard memory	Only for the "unreachable, last seen …" view. Lost on restart, shown as "not contacted yet".

Behaviour when a node is unreachable

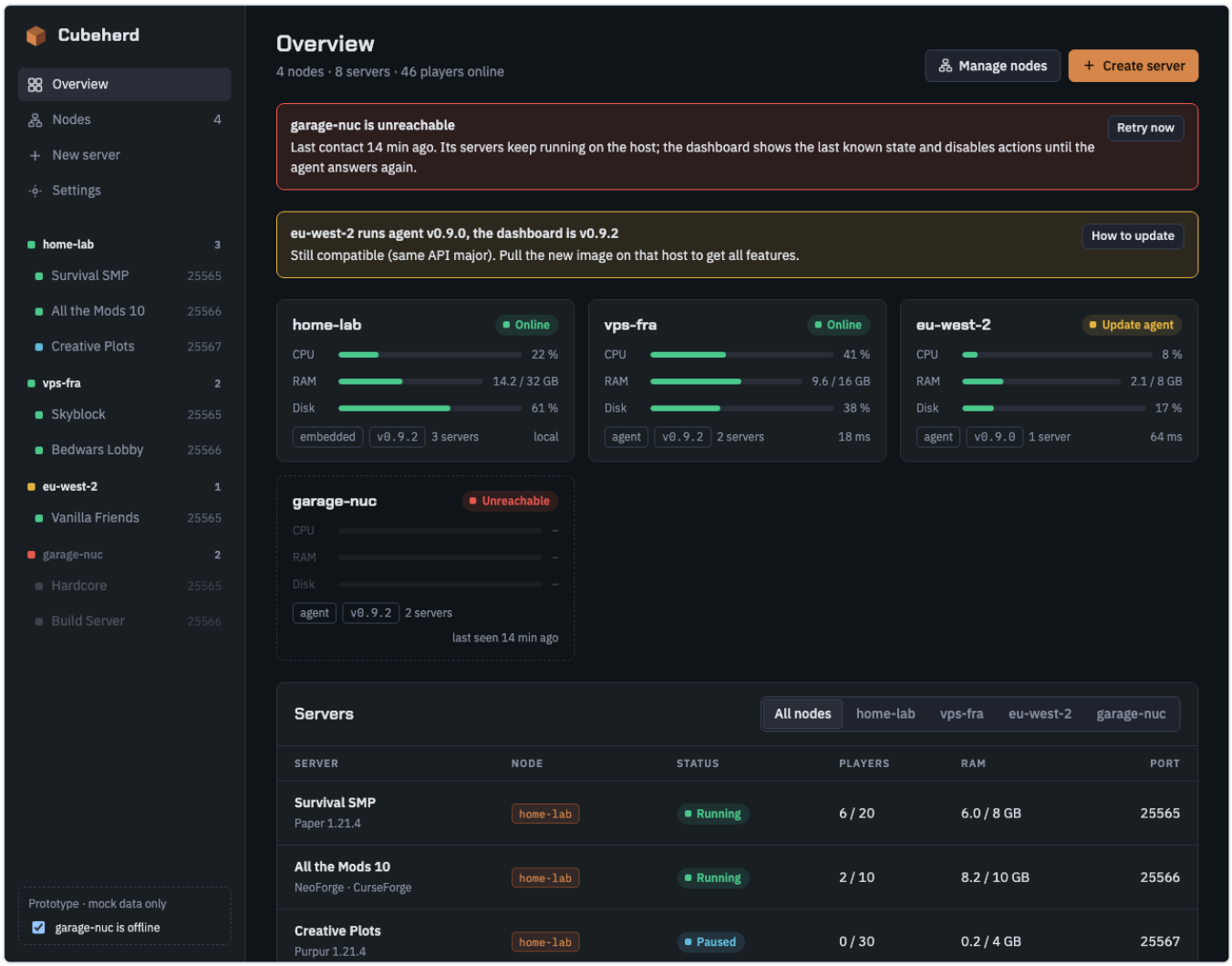
- The node card turns red, its servers are dimmed with their last known state, and all actions on them are disabled. The servers themselves keep running; so do their scheduled backups.
- Reads time out after 2 s so one dead node never slows the overview. Nodes are queried in parallel, and only on page views.
- Removing a node never touches its servers or data. Re-adding it restores the full view.

Compatibility with existing installs

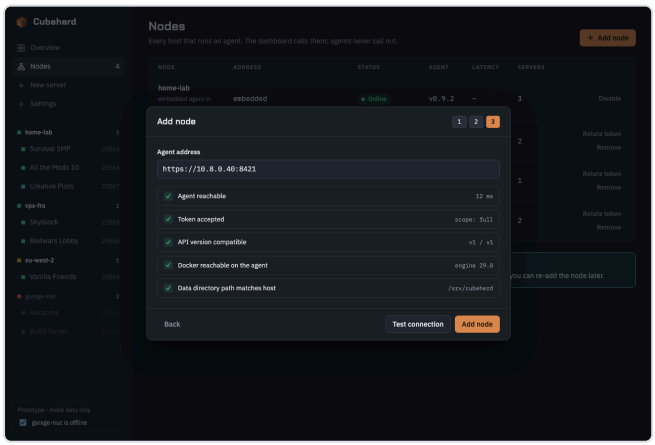
- The data directory format does not change. On first start the dashboard creates an implicit **local** node (named after the host) when the Docker socket is present, so an existing single-host setup looks and behaves the same.
- The documented `compose.yml` keeps working. A dashboard-only variant (no socket) and an agent variant are added next to it.
- Old `/s/<id>` URLs redirect to the local node.

6. The interface (prototype)

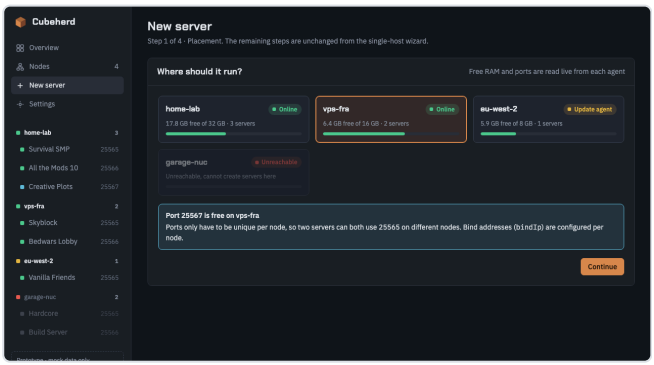
The clickable prototype is `prototype/nodes.html`. It uses mock data and the existing visual language. A toggle in the sidebar switches the offline scenario.



Overview. One card per node (status, CPU, RAM, disk, agent version, latency), a banner for an unreachable node and for an outdated agent, and a server table with a node column and filter. The sidebar groups servers by node.

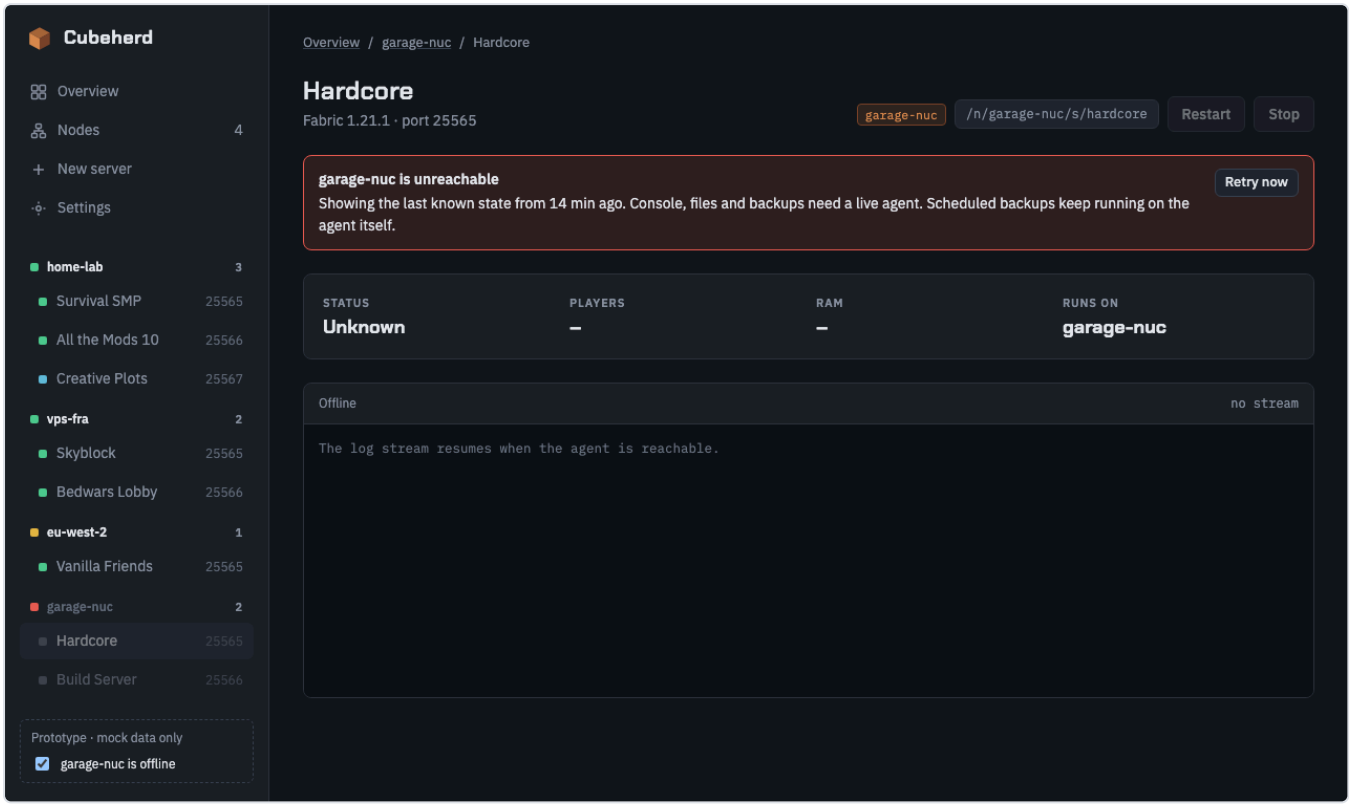


Add node. Three steps: name, run the agent snippet on the host, test the connection (reachable, token, API version, Docker, data path).



Placement step. The create wizard asks where the server runs; free RAM is read live, offline nodes are disabled, and the port is checked per node.

6. The interface (continued)



Server detail on an unreachable node. Breadcrumb and URL carry the node. Actions are disabled, the console explains why, and the banner states that scheduled backups keep running on the agent.

UI changes in short

- **Sidebar:** servers grouped by node with a status dot per node; offline nodes dimmed.
- **Overview:** node cards, node chip and filter in the server table. Totals (players, servers) sum over reachable nodes.
- **Nodes (new):** table, add dialog, token rotation, removal, version and latency.
- **New server:** placement step first; the remaining steps are unchanged.
- **Server page:** node chip, breadcrumb, `/n/<node>/s/<id>` URL, stream source shown in the console header.
- **i18n:** every new text needs a German source string and an English entry; `TestCatalogsComplete` enforces both directions.

7. Delivery plan

Phase 1	**Node interface, no behaviour change** Introduce `internal/node`, wrap the Manager as the local node, route every handler through it, remove direct `Store` access from the web layer. All existing tests stay green; the product looks identical.
Phase 2	**Agent mode and remote node** Agent HTTP server and the remote client, token and certificate pinning, streaming for events, files, world and backups. Tested by running an agent and a dashboard against each other.
Phase 3	**Dashboard UI** Node registry, Nodes page, grouped sidebar and overview, placement step, new URL scheme with redirects, offline handling, i18n strings.
Phase 4	**Packaging and docs** Dashboard and agent compose files, Docker image entrypoint, README and ARCHITECTURE.md, migration notes, release notes.
Later	**Optional extensions** mTLS instead of token plus pinning; moving a server between nodes (backup, restore elsewhere); an agent-initiated tunnel for hosts behind NAT; read-only tokens.

Risks and open points

RISK	MITIGATION
Large transfers through the dashboard	Pure streaming proxy with no buffering; resume is out of scope for the first version.
Token and S3 secrets stored on several hosts	Mode 0600, tokens hashed on the agent, rotation from the UI, documented VPN-only exposure.
Clock or time-zone drift between hosts	The dashboard pushes one time zone; agents use it for schedules and container `TZ`.
Version skew	API major must match; the dashboard flags older agents and refuses incompatible ones with a clear message.
Same itzg container adopted twice	Discovery runs per node, so a container is only ever visible on the node that hosts it.

Next step
After review of this document and the prototype, start with Phase 1. It is a pure refactor and can ship on its own.